

GENAI PINNACLE PLUS PROGRAM

# CAPSTONE PROJECT

---

## Research Paper Answer Bot

*RAG • Vector Databases • LLMs • LangChain*

**Build an intelligent chatbot over research papers  
using RAG**

Duration: 30 days • Total Marks: 100 • Submission: Code + PPT

# 1. Project Overview

| Field             | Details                                            |
|-------------------|----------------------------------------------------|
| Project Title     | Research Paper Answer Bot                          |
| Core Technologies | RAG, Vector Databases, LLMs, LangChain, Embeddings |

## 2. Business Case & Problem Statement

You are working for a company like [ArXiv](#) which has a curated set of important research papers around AI and Data Science.

The broad idea of this project is to build a RAG system on top of a curated set of important research papers around Generative AI and LLMs. This system should be able to:

- ☐ Index the research papers into a vector database
- ☐ Use good retrieval strategies to retrieve relevant contextual documents based on input queries
- ☐ Generate accurate, grounded responses citing supporting passages

The focus of this project is not just to build a simple RAG system, but to explore various approaches and methodologies in each component — and then choose the approach that works best.

**Note:** You are not compelled to use the sample data provided. If you have your own documents or data, you are welcome to use them — but validate your data and idea with your mentor first. The project steps mentioned below remain unchanged regardless of the dataset used.

## 3. Project Goals

The project is divided into Compulsory Goals (must complete all) and Stretch Goals (complete at least 1).

### 3.1 Compulsory Goals

You must complete ALL of the following:

1. Get your own dataset or [download the provided PDFs of seminal GenAI research papers](#).

2. Load the files and index them in a vector database.
3. Experiment with different embedding models — open-source models from Hugging Face and commercial ones (e.g., OpenAI embeddings).
4. Experiment with various retrieval strategies — from simple cosine similarity to hybrid search and rerankers.
5. Connect your vector database to an LLM and build a complete RAG pipeline.
6. Test the RAG pipeline on sample queries relevant to the research papers.
7. Show the top-3 sources used to generate the response, including paper title and page number.

### 3.2 Stretch Goals (Complete At Least 1)

You must complete at least one of the following advanced options. Completing more is encouraged and will be rewarded:

- ☐ **Advanced Option 1 (Medium):** Enhance the system into a multi-user conversational RAG system with chat history and memory. [See LangChain tutorial](#)
- ☐ **Advanced Option 2 (Medium):** Build a Streamlit or Chainlit application on top of your RAG system for a polished, shareable UI.
- ☐ **Advanced Option 3 (Hard):** Enhance your system with web-search using Agentic Corrective RAG (CRAG) patterns. [Read the Analytics Vidhya reference article](#)

## 4. Project Milestones

The following milestones provide a structured plan for completing the project:

| Milestone                  | Key Tasks                                                                                                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 — Load & Index Documents | Download/collect research PDFs. Parse and chunk documents. Set up vector database. Index chunked documents.                                                   |
| 2 — Embeddings & Vector DB | Experiment with at least 2 embedding models (open-source + commercial). Compare embedding quality. Finalise embedding model and vector store.                 |
| 3 — Retrieval Strategy     | Implement simple cosine similarity retrieval. Try hybrid search (keyword + semantic). Experiment with rerankers. Evaluate and select best retrieval approach. |

|                        |                                                                                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 — Basic RAG System   | Connect retriever to an LLM. Build the RAG chain. Test on sample queries. Display top-3 supporting passages per answer, including paper title and page number. |
| 5 — Enhanced RAG / App | Implement at least 1 stretch goal. Polish the pipeline. Prepare demo and presentation.                                                                         |

## 5. Suggested Approach & Technical Steps

The following is a recommended technical pipeline. You are encouraged to experiment beyond these suggestions.

### Step 1 — Data Collection & Document Loading

1. Collect research PDFs (provided dataset or your own validated data).
2. Use document loaders (e.g., LangChain's PyPDFLoader, PDFPlumberLoader) to extract text.
3. Inspect document structure — page count, text quality, and any metadata.
4. Handle edge cases: scanned PDFs may need OCR.

### Step 2 — Text Chunking Strategy

1. Experiment with different chunking approaches:
  - Fixed-size chunking (e.g., 512 tokens with overlap)
  - Recursive character text splitter
  - Semantic chunking (split on topic shifts)
2. Choose chunk size and overlap that preserves context without excessive noise.
3. Retain metadata per chunk (source file, page number) — needed for source citation.

### Step 3 — Embedding Models

1. Experiment with at least two embedding models:

| Type                       | Examples                                                                 |
|----------------------------|--------------------------------------------------------------------------|
| Open-Source (Hugging Face) | BAAI/bge-m3, Qwen/Qwen3-Embedding-4B, mixedbread-ai/mxbai-embed-large-v1 |
| Commercial                 | OpenAI text-embedding-3-small, OpenAI text-embedding-3-large             |

2. Compare embeddings qualitatively on a few sample queries.

3. Document your observations and justify your final choice.

## Step 4 — Vector Database

1. Set up a vector database to store and query embeddings:
  - Recommended options: ChromaDB (local) or FAISS (local). You may use another vector store if justified.
2. Index all document chunks with their embeddings and metadata.
3. Validate that retrieval returns semantically relevant chunks.

## Step 5 — Retrieval Strategies

1. Implement and compare multiple retrieval approaches:

| Strategy                     | Description                                                | Complexity  |
|------------------------------|------------------------------------------------------------|-------------|
| Dense Retrieval (Cosine)     | Baseline vector similarity search using embeddings.        | Low         |
| MMR (Max Marginal Relevance) | Balances relevance and diversity in retrieved chunks.      | Low         |
| Hybrid Search                | Combines keyword (BM25) and dense retrieval scores.        | Medium      |
| Rerankers                    | Cross-encoder reranking of top-k candidates for precision. | Medium-High |
| Multi-query Retrieval        | LLM generates multiple query variants to improve recall.   | Medium      |

2. Evaluate retrieval quality using a set of test queries.
3. Select the strategy that gives the best precision and recall trade-off.

## Step 6 — RAG Pipeline Construction

1. Connect the retriever to an LLM. Suggested model choices: OpenAI GPT-4.1 mini (API) for a strong hosted baseline, or an open-source model through Ollama such as Llama 3.1 8B for local inference.
2. Design an effective prompt template — include retrieved context and question.
3. Build the RAG chain using LangChain, LCEL, or a custom chain.
4. Ensure the pipeline returns the top-3 supporting passages with each answer.

### Key Prompt Engineering Tips:

- Instruct the model to answer ONLY from the provided context.
- Ask the model to say 'I don't know' if context is insufficient — avoids hallucination.
- Include source paper title, page number, and chunk metadata in the prompt or response formatting for grounded citations.

## Step 7 — Testing & Evaluation

1. Prepare a set of at least 10 test questions spanning different papers.
2. Evaluate responses for: relevance, accuracy, and groundedness.
3. Document failure cases — where does the system struggle?
4. Consider using automated evaluation with DeepEval or another LLM-as-a-judge setup to score answer correctness, groundedness, contextual relevancy, and hallucination risk.

## Step 8 — Stretch Goal Implementation

- ☐ Option 1: Add conversational memory or chat history handling to support multi-turn question answering.
- ☐ Option 2: Build a Streamlit / Chainlit UI with query input, response display, and source viewer.
- ☐ Option 3: Implement Corrective RAG — if retrieval confidence is low, trigger a web search fallback.

## 6. Deliverables

You are required to submit the following:

### 6.1 Code File (Jupyter Notebook / Python Script)

- ☐ A well-structured, reproducible Jupyter Notebook (.ipynb) or Python script(s).
- ☐ Runs end-to-end without errors.
- ☐ Clearly organised into sections matching the approach steps above.
- ☐ Markdown cells explain each step, model choices, and observations.
- ☐ All experiments (embedding models, retrieval strategies) documented inline.
- ☐ Final RAG pipeline produces answers with top-3 supporting citations, including paper title and page number.

## 6.2 Presentation (PowerPoint / PDF)

A presentation (10–15 slides) covering:

| Slide # | Section                | Content                                                        |
|---------|------------------------|----------------------------------------------------------------|
| 1       | Title Slide            | Project title, your name, program name, date                   |
| 2       | Problem Statement      | Business context, objective, and use case                      |
| 3       | Architecture Overview  | End-to-end RAG architecture diagram                            |
| 4–5     | Data & Chunking        | Dataset details, chunking strategy chosen and why              |
| 6       | Embeddings             | Models compared, observations, final choice with justification |
| 7       | Vector DB & Retrieval  | Database used, retrieval strategies compared, final selection  |
| 8       | RAG Pipeline           | Chain construction, prompt template, LLM used                  |
| 9       | Results & Demo         | Sample Q&A outputs with source citations shown                 |
| 10      | Stretch Goal           | What was built, how it enhances the system                     |
| 11      | Challenges & Learnings | Key difficulties and how they were resolved                    |
| 12      | Conclusion             | Summary and potential future improvements                      |

## 6.3 Live Demo

- ☐ A live walkthrough of your RAG system during the final mentor review.
- ☐ Demonstrate at least 5 queries with responses and supporting passages shown.
- ☐ If you built a Streamlit/Chainlit app (Stretch Option 2), demo the UI.

## 7. Evaluation Rubrics (Total: 100 Marks)

## 7.1 Project Component (60 Marks)

| Sub-Category                             | Marks | What Evaluators Look For                                                                                                                                                                                                                    |
|------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem Understanding & Data Preparation | 15    | Clear understanding of the RAG use case. Proper document loading, text extraction, and chunking strategy. Metadata retention for source citations. Clean, well-documented data pipeline.                                                    |
| Vector Database & Embedding Strategy     | 15    | At least two embedding models compared with observations. Appropriate vector database selected and set up correctly. Embeddings indexed with metadata. Justification for final embedding choice.                                            |
| Retrieval Strategy                       | 15    | Multiple retrieval strategies implemented and compared. Clear evaluation methodology. Final retrieval approach selected with rationale. Supporting passages correctly retrieved and attributed, with paper title and page number.           |
| RAG Pipeline                             | 15    | Complete end-to-end RAG chain built. Effective prompt engineering. LLM connected and generating grounded responses. Top-3 supporting passages shown per answer, including paper title and page number. At least 1 stretch goal implemented. |

## 7.2 Technical Understanding (40 Marks)

Evaluated during the final project walkthrough and Q&A:

| Area               | Marks | Topics Covered                                                                                                        |
|--------------------|-------|-----------------------------------------------------------------------------------------------------------------------|
| Python             | 10    | Code quality, use of relevant libraries (LangChain, Hugging Face, OpenAI SDK), efficient and readable implementation. |
| Prompt Engineering | 10    | Understanding of prompt design for RAG, system/user prompts, context injection, hallucination prevention strategies.  |



|           |    |                                                                                                                               |
|-----------|----|-------------------------------------------------------------------------------------------------------------------------------|
| RAG       | 10 | Conceptual understanding of RAG architecture, chunking trade-offs, retrieval precision vs recall, grounding and faithfulness. |
| LangChain | 10 | Use of LangChain components (loaders, splitters, vectorstores, retrievers, chains, memory), LCEL pipeline construction.       |

| Component                                                                                    | Marks      |
|----------------------------------------------------------------------------------------------|------------|
| Project (Problem Understanding + Vector DB & Embeddings + Retrieval Strategy + RAG Pipeline) | 60         |
| Technical Understanding (Python + Prompt Engineering + RAG + LangChain)                      | 40         |
| <b>TOTAL</b>                                                                                 | <b>100</b> |

## 8. Tips, Best Practices & Common Mistakes

### Best Practices

- ☐ Start with a small subset of documents to build and validate your pipeline before scaling.
- ☐ Always store chunk metadata (source file, page number) at indexing time — you'll need it for citations.
- ☐ Test your retriever independently before wiring it to the LLM — retrieval quality is the biggest bottleneck.
- ☐ Use a separate evaluation set of queries to compare embedding models and retrieval strategies objectively.
- ☐ Prompt your LLM to stay grounded: instruct it to answer only from provided context.
- ☐ Version your experiments — log embedding model, chunk size, retrieval strategy, and qualitative results in a table.

### Common Mistakes to Avoid

- ☐ Using a single embedding model without comparison — always experiment with at least two.
- ☐ Ignoring chunk overlap — too little causes context loss at boundaries; too much inflates index size.
- ☐ Not retaining metadata during chunking — makes source citation impossible later.

- ☐ Evaluating only on queries the system was built around — test on genuinely unseen questions.
- ☐ Over-relying on LLM output quality without validating the retrieval step first.

## Recommended Libraries & Tools

| Purpose                   | Recommended Libraries / Tools                                              |
|---------------------------|----------------------------------------------------------------------------|
| Document Loading          | LangChain (PyPDFLoader, PDFPlumberLoader), pypdf, pdfplumber               |
| Text Splitting            | LangChain (RecursiveCharacterTextSplitter, SemanticChunker)                |
| Embeddings                | sentence-transformers, openai, langchain-huggingface                       |
| Vector Databases / Stores | ChromaDB, FAISS, Pinecone, Qdrant, Weaviate                                |
| Retrieval                 | LangChain retrievers, rank_bm25 (hybrid), sentence-transformers (reranker) |
| LLM Integration           | LangChain, OpenAI SDK, Anthropic SDK, Ollama (local)                       |
| RAG Evaluation            | RAGAS, LangChain evaluation modules, DeepEval                              |
| App / UI                  | Streamlit, Chainlit                                                        |
| Web Search (CRAG)         | Tavily API, DuckDuckGo search, SerpAPI                                     |

## 9. Submission Checklist

Use this checklist before submitting to ensure all components are complete.

### Code

- ☐ Notebook runs end-to-end without errors
- ☐ At least 2 embedding models compared with documented observations
- ☐ Vector database set up and all documents indexed
- ☐ At least 2 retrieval strategies implemented and compared

- ☐ Complete RAG chain built with LLM integration
- ☐ Top-3 supporting passages returned with each answer, including paper title and page number
- ☐ At least 1 stretch goal implemented

## Presentation

- ☐ 10–15 slides covering all required sections (see Section 6.2)
- ☐ RAG architecture diagram included
- ☐ Embedding and retrieval comparison documented
- ☐ Sample Q&A outputs shown with citations

## Demo

- ☐ At least 5 live queries demonstrated during mentor review
- ☐ Supporting passages displayed alongside each answer

## Good luck!

*This capstone is designed to test your ability to build a production-quality RAG system from the ground up. Focus on understanding each component deeply, experimenting rigorously, and building something you are proud to demo. Quality of thinking and engineering process matters as much as the final output.*